

Session 5: Subagents, Hooks, and Final Project

Spawning focused agents, event-driven automation, and an end-to-end research artifact

Brady Allardice

UPF & IBEI

Start-of-Session Ritual

Same three steps as last session, with today's folder.

```
# 1. Pull the latest course repo
cd ~/claude-code-workshop && git pull

# 2. Copy today's session folder into your workspace
cp -r session_5/ ~/my_workspace/session_5/

# 3. Open your copy in VS Code and work there
cd ~/my_workspace/session_5 && code .
```

Don't have the course repo yet?

```
git clone https://github.com/bradyallardice/claude-code-workshop.git
```

Five minutes, together, live. Everyone in a clean state before we move on.

If `git pull` fights back, use the recovery from Session 4: `git stash -u`, `git branch backup-s5`, `git fetch origin`, `git reset --hard origin/student`, `git stash pop`.

Today: Agents, Hooks, and a Paper

Three parts.

- 1 **Agents.** Subagents, named specialists, and multi-agent orchestration — when to spawn one, how to define your own, how they compose.
- 2 **Hooks.** Shell commands that run automatically on events (file writes, tool use, session start) — enforcement that doesn't depend on Claude remembering.
- 3 **Putting it together.** Use today's pieces (plus everything from Sessions 1–4) to build the spine of a research paper end-to-end.

The earlier sessions taught the moves. This one is about composing them into something you'd actually submit.

A focused agent spawned for a bounded job —
the right tool for the parallelizable, scoped work
that already shows up in your pipeline.

Concrete patterns, then named teams.

What a Subagent Is

Claude Code can spawn another Claude Code instance inside your current session — a subagent.

- It has its own context window
- It gets a scoped task and a scoped tool set
- It returns a summary to your main conversation — not its raw trace

What that buys you:

- ➊ **Parallelism** — run three searches at once instead of one after the other
- ➋ **Context hygiene** — a big messy exploration stays out of your main context budget

Think of it as

Delegating a bounded sub-task to a focused colleague who returns a clean, structured answer — so your main conversation stays on the high-level work.

Where Subagents Shine

Reach for a subagent when the subtask is:

- **Parallelizable** — independent work that doesn't need to be sequenced
- **Bounded** — clear inputs, clear outputs, no back-and-forth
- **Independently verifiable** — you can check the result without reading the full trace

Concrete research use cases:

- **Multi-database literature searches** — one agent per database, all running in parallel
- **Coding many documents** against a fixed rubric
- **Parallel robustness checks** across independent specifications
- **Running the same sanity-check** script over many datasets
- **Exploring a codebase** — send a subagent to map out the files, keep the summary in your main context

How to Trigger a Subagent

You don't have to set anything up — just ask.

Claude Code ships with a general-purpose subagent built in. To spawn one, describe the work in plain English:

- *“Send a subagent to map the directory structure”*
- *“Use a subagent to find every place we set the random seed”*

For parallelism, say so explicitly.

- *“Spawn three agents **in parallel** to search Zotero, Google Scholar, and Connected Papers”*
- “In parallel” / “at the same time” / “concurrently” all signal that the agents should run in one batch rather than sequentially

No setup required

The general-purpose agent ships with Claude Code. Defining your own named agent (later this session) is for when you want to reuse a scoped role across many sessions.

Using a Generic Subagent in Practice

Worked example: one subagent, one bounded local task.

Your prompt to Claude:

```
Spawn a subagent to read the slides for sessions 1-4  
and return a deduplicated list of every unique terminal  
command I'll need across the course.
```

```
Output as: command | what it does | first session  
it appears in.
```

What Claude does:

- Spawns one general-purpose subagent and hands it the four .tex files
- The subagent does the reading, extraction, and dedup in its own context
- You get back the table; the slide text never lands in your main conversation

Three habits that make subagents pay off:

- ① **Brief them like a contractor.** Concrete deliverable, all the context they need up front — they don't see your conversation history.
- ② **Ask for structured output** (bullet list, table, JSON). You'll be consuming the result; make it easy to read and verify.
- ③ **Verify independently.** You get a summary, not the full trace. Spot-check the way you would a student's work.

The payoff

A literature sweep, robustness battery, or repo survey that would have taken an hour finishes in minutes — and doesn't eat into your main conversation's context. That's what makes them worth reaching for.

What a Predefined Subagent Looks Like

A small markdown file — author it once, Claude spawns it many times.

Where it lives: `~/.claude/agents/<name>.md` (user-level) or
`.claude/agents/<name>.md` (project-level).

What you specify (metadata + body):

- **Identity** — name and a one-line description. The description is what Claude reads to decide when to spawn this agent.
- **Tool allowlist** — exactly which tools it can use (e.g. Read only, no Bash).
- **Model** — Opus, Sonnet, or Haiku.
- **System prompt** — the body of the file: role, disposition, expected output format.

Plain text — metadata sits between two `---` lines at the top of the file; the system prompt is everything below.

Building a Specialized Agent

Worked example: turn “audit my regression script” into a reusable role.

Walking through the four fields:

- 1 **Name + description.** What it does, plus the one-line trigger Claude reads to decide when to spawn it.
“Audit a regression script for silent issues.”
- 2 **Tool allowlist.** Read, Grep, Glob — read-only. A critic that can fix code is no longer a critic.
- 3 **Model.** haiku — fast and cheap, fine for systematic pattern-matching.
- 4 **System prompt.** Disposition + output contract: *“You’re a skeptical reviewer. Flag invalidating issues. Return: line, issue, why. Don’t propose fixes.”*

Save as `~/.claude/agents/spec-critic.md`.

Now in any session:

- Claude auto-spawns it when a task matches the description
- Or invoke explicitly: *“use spec-critic on solution_estimate.py”*
- Role is enforced — if it suggests a fix, it broke its own contract

Permissions still apply: tools the agent uses must be allowed in `.claude/settings.json`.

Why Define a Named Subagent at All?

If skills are reusable, why not just spawn ad-hoc subagents that pick up the right skill?

The framing

Skills are knowledge. Subagents are roles.

A role bundles a disposition, a tool scope, and an invocation contract — and then draws on skills to execute.

Three things a role gives you that a skill can't:

- ❶ **Tool scope — enforced.** A code-review subagent has read-only access, no bash. Skills are advisory; tool scopes aren't.
- ❷ **Identity, not advice.** The system prompt frames the whole disposition — “find problems, don't fix them.” Subagents *are* the disposition.
- ❸ **A stable invocation contract.** Delegate code review → get a structured critique. Ad-hoc means re-specifying the contract each time.

Pre-define a subagent when the **role** is the reusable unit, not just the know-how.

When one specialist isn't enough.
Compose subagents into pipelines, fan-outs,
and critic-builder loops.

Three patterns, one worked example.

Three Composition Patterns

Most multi-agent workflows are one of three shapes.

- ① **Fan-out / map-reduce.** N inputs, one specialist per input, merge at the end.
Example: code 50 interview transcripts against the same rubric — one agent per transcript, then synthesize.
- ② **Pipeline.** Each stage's output is the next stage's input. Named and generic agents can mix.
Example: spec-critic flags issues → generic agent drafts fixes → spec-critic re-audits.
- ③ **Critic-builder loop.** One agent produces, another reviews, the producer revises.
Example: a paragraph drafter ↔ a paragraph-reviewer, two or three rounds.

A Pipeline in Practice

Worked example: hardening a regression script in three stages.

Your prompt to Claude:

Run this pipeline on `solution_estimate.py`:

1. Use `spec-critic` to list every silent issue.
2. Spawn a subagent to draft minimal-diff fixes for each issue.
3. Re-run `spec-critic` on the patched script.

Return: original issues | proposed fixes | residual issues.

What makes this work:

- Each stage has a stable input/output contract — so they compose
- Two `spec-critic` calls share a role but run in fresh contexts — consistent without state
- Your main conversation only sees the final summary; intermediate traces stay isolated

One Agent or Many?

Default to one agent. A team has to earn its keep.

Stick with one agent when:

- The task is bounded enough to fit one context window
- One disposition is enough — no producer/critic split
- You're still exploring — you don't yet know the right stages
- Orchestration overhead would dominate the actual work

Reach for a team when:

- The work has naturally distinct roles (writer + reviewer; auditor + fixer)
- Independent verification is the point — a separate set of eyes
- You'll run the workflow many times — the setup cost amortizes
- N independent inputs need the same treatment in parallel

The test

Each extra agent is a stage that can drift. Add one only if it does something the previous one demonstrably can't.

Shell commands the harness runs automatically
on events — file writes, tool calls, session start.
Enforcement that doesn't depend on Claude remembering.

Concept, one example, when to reach for one.

What a Hook Is

A hook is a shell command wired to a Claude Code event. Configured in `.claude/settings.json`; run by the harness, not by Claude.

Common events:

- **PreToolUse** — before a tool runs (can block it)
- **PostToolUse** — after a tool runs
- **SessionStart** — at the start of a session
- **UserPromptSubmit** — when you submit a prompt
- **Stop** — when Claude finishes responding

What this buys you:

- **Enforcement, not advice.** The hook runs whether Claude remembers it or not.
- **Auditability.** Log every tool use, every edit, every Bash call.
- **Automation.** Format on save, refresh derived files, notify on completion.

Worked Example: A Research Audit Log

Goal: every Bash command Claude runs gets logged with a timestamp — automatically, every session.

Add to `.claude/settings.json`:

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "echo \"$(date '+%F %T') $CLAUDE_TOOL_INPUT\" >> ~/research_log.txt"
      }]
    }]
  }
}
```

What happens: Before every Bash call, the harness runs the command above and appends a timestamped line to your research log. Claude can't bypass it — the harness, not the model, runs hooks.

Hook, Skill, or Agent?

Three layers of automation, three different things.

The framing

Skills are knowledge. Subagents are roles. Hooks are guarantees.

Reach for a:

- **Skill** when Claude needs to know *how* to do X.
- **Subagent** when you want a delegated worker with a scoped role.
- **Hook** when something must happen regardless of what Claude remembers.

Hooks are the only one of the three the harness enforces. Skills are advice; subagents have agency; hooks are mechanical.